RESEARCH ios_android

RESEARCH ios_android

Revision: 1 **Last modified:** 2026-07-04T12:00:00Z

Self-hosted Mullvad-parity VPN — mobile native core constraints. Deep multi-angle research (§11.4.150 / §11.4.99). Access date for every source: **2026-06-25**. All numeric claims cited; where a figure is unofficial/undocumented it is marked.

---

1. iOS NEPacketTunnelProvider memory limit — current reality

FACT — current limit is 50 MiB (iOS 15+ through iOS 18)

Apple DTS engineer Quinn "The Eskimo!" published a measured table (Oct 2022, iOS 16.0) giving the Packet Tunnel Provider limit as **50 MiB**, and confirmed (Sep 2024 update on the same thread) it still applies to iOS 17 and iOS 18.

- Historical progression (per Quinn + DTS threads):
  - iOS ~10 beta: ~5–6 MB → raised to ~15 MB
  - iOS 11–14 (64-bit): **15 MiB**
  - iOS 15: raised to **50 MiB**
  - iOS 16 / 17 / 18: **50 MiB** (measured)
- Source: https://developer.apple.com/forums/thread/73148 (Quinn, "packet tunnel | 50" table, iOS 16.0; reconfirmed for 17/18)

CRITICAL CAVEAT — the limit is NOT documented and MUST NOT be hardcoded

Quinn, verbatim:

> "These limits have changed in the past and may well change in the future. … You should not hard code knowledge about these limits into your code. The only way to ensure that your provider can run within the system's memory limits is to thoroughly test it on a wide range of device and OS combinations." He also warned the 50 MiB was measured on a "very modern device" and **limits may be lower on older devices**.

- Source: https://developer.apple.com/forums/thread/73148
- Source: https://developer.apple.com/forums/thread/106377 (Quinn: "the exact limit is not officially documented")

CONTRADICTION / real-world risk — observed 15 MB kills on iOS 17 (UNCONFIRMED root cause)

A developer (Feb 2024, iPhone 14 Pro Max, iOS 17.3.1) reported the extension was still killed by jetsam at ~15 MB despite the documented 50 MiB, while running `tun2socks` forwarding to a SOCKS5 (xray) port. Apple DTS did not explain the discrepancy on-thread.

- PENDING_FORENSICS: whether the 15 MB kill is a true lower limit on some devices, a measurement artifact, or peak-vs-resident accounting. Do NOT assume 50 MiB headroom.
- Source: https://developer.apple.com/forums/thread/747474

Scope of the limit

The limit applies to the **whole extension process** — your code AND every linked library (Rust staticlib, third-party TLS, tun2socks, etc.) count against it.

- Source: https://developer.apple.com/forums/thread/106377 ("applies to the process as a whole, and thus to your code and any library code you use")

Practical headroom guidance (engineering conclusion)

- WireGuard-class designs fit easily: WireGuard operates at the IP-packet level with no TCP connection tracking, so persistent heap state is minimal.
  - Source: https://developer.apple.com/forums/thread/73148 (Quinn: "most VPN implementations are kinda simple")

- Memory spikes (not steady-state) are the killer: `sing-box`/WireGuard users see jetsam termination during **upload-heavy speedtests**, i.e. transient buffer-pool / GC spikes.
  - Source: https://github.com/SagerNet/sing-box/issues/3976 (iOS extension memory crash during SpeedTest on WireGuard connections)
- Design rule: target a steady-state budget well under the unofficial floor (plan to a ~12–15 MB working set, not 50 MiB), bound packet buffer pools, avoid per-flow state growth, prefer a single fixed-size receive buffer (see "max datagram size" below).

How to measure on device (no public API for the limit itself)

- There is no API to query or raise the NE memory limit; you must measure footprint and test empirically (Quinn). Sources do not endorse a specific footprint API on-thread, but the established practice is:
  - Instruments "Allocations" / "VM Tracker" + the Memory gauge in Xcode against the extension process (debug-attach to the running NE target).
  - For a Go/Rust core, log runtime allocation stats periodically to find spikes — Go devs log `runtime.ReadMemStats` from inside the extension to catch allocation spikes.
    - Source: https://groups.google.com/g/traffic-obf/c/PksmyfHMUb4 (Using Go in Apple network extensions)
  - `task_vm_info` / `phys_footprint` (Mach) is the value jetsam accounts against (general iOS knowledge; not quoted from a source above — verify on device).
- Max received datagram size in the packet tunnel is itself a tuning question (bound your read buffer to the negotiated MTU + headroom, not an arbitrary large value).
  - Source: https://developer.apple.com/forums/thread/680486 (Max received datagram size in iOS packet tunnel)

---

2. Rust staticlib size optimization for iOS

Goal: shrink the static archive linked into the NE (both for app-size and to keep the extension's resident footprint low — smaller code, fewer pages mapped).

Baseline Cargo `[profile.release]` (size-first)

```
[profile.release]
opt-level = "z"     # smallest code (try "s" too — often faster
        with ~similar size)
```

```
lto = true          # whole-program; "fat" for max size reduction
codegen-units = 1   # better cross-module size opt (slower
        compile)
panic = "abort"     # drops unwinding tables + landing pads
strip = true        # strip symbols (Rust 1.59+); ~3–8% extra
```

- `panic = "abort"` removes stack-unwinding info → smaller + sometimes faster.
- `strip = true` removes function names/debug info, ~3–8% additional reduction.
- Sources: https://doc.rust-lang.org/rustc/codegen-options/index.html ; https://nnethercote.github.io/perf-book/build-configuration.html

Advanced: build-std with size features (biggest wins)

Rebuild the standard library for the iOS target with size features:

- `cargo +nightly build -Z build-std=std,panic_abort -Z build-std-features=panic_immediate_abort,optimize_for_size`
- Effect: drops large chunks of `std` (all unwinding/panic-formatting machinery incl. `gimli` backtrace), and lets `std` itself be compiled for size.
- Representative iOS flag set seen in practice: `-C panic=abort -C opt-level=s -C lto=fat -C codegen-units=1` (plus `--inline-threshold` tuning).
- Source: https://markaicode.com/binary-size-optimization-techniques/ (reports ~43% reduction; build-std + panic_immediate_abort + optimize_for_size)

Mobile-SDK-specific lessons (real engineering blog)

bitdrift documented shrinking a Rust mobile SDK: dominant wins were LTO + codegen-units=1

- panic=abort + stripping, then `build-std`/`optimize_for_size`; also watch monomorphization bloat (generic explosion) and prefer `dyn` where hot-path cost allows; audit fat dependencies (regex, formatting, async runtimes) that pull in large code.

- Source: https://blog.bitdrift.io/post/optimizing-rust-mobile-sdk-binary-size

staticlib vs dylib on iOS

- `crate-type = ["staticlib"]` produces a `.a` to link into the NE/app (preferred for iOS: no dynamic-loading entitlement friction; symbols dead-stripped by the linker).

- Practical guide to building Rust for iOS (and the dylib alternative + bridging):
  - Source: https://ospfranco.com/complete-guide-to-dylibs-in-ios-and-android/
  - Source: https://ospfranco.com/rust-reduce-binary-size/

---

3. Android VpnService — constraints, JNI native core, fd handoff

Establishing the tunnel (Layer-3 fd)

- `VpnService.Builder` configures the interface: `addAddress()`, `addRoute()`, `addDnsServer()`, `setMtu()`, `setSession()`, then `establish()` returns a `ParcelFileDescriptor` for the TUN interface.
- The app **reads IP packets from** and **writes IP packets to** that fd — Layer 3 (raw IP), not Layer 4. There is no per-socket model; it's a single packet pipe.
- Builder MUST set at least one address and the routes/MTU before `establish()`.
- Sources: https://developer.android.com/reference/android/net/VpnService.Builder ; https://developer.android.com/reference/android/net/VpnService

Handing the fd to native (Rust/C) core via JNI

- Get the raw int fd with `ParcelFileDescriptor.detachFd()` (transfers ownership; the native side now owns close()) OR `getFd()` (keeps ownership in Java).
- Pass that int into JNI; the native core does `read(2)`/`write(2)` directly on it (this is exactly how tun2socks-style cores consume the Android tun fd).
- Source: https://github.com/xjasonlyu/tun2socks/issues/123 (using the file descriptor in native space; detachFd)
- Source: https://medium.com/@bvenom87/building-a-minimal-custom-vpn-in-android-from-tun-interfaces-to-real-time-status-4847e6e382a1

protect() — the loopback-avoidance requirement (mandatory)

- `VpnService.protect(int socket)` / `protect(Socket)` binds a socket to the **underlying physical network**, so the tunnel's own outbound packets do NOT get routed back into the tun (which would infinite-loop).

- MUST be called **before** the socket connects/sends. Every outbound transport socket the native core opens (to the VPN server) must be protected.
- Source: https://developer.android.com/reference/android/net/VpnService

Calling protect() directly from native via JNI (perf / architecture note)

- `VpnService.protect()` ultimately calls the C function `protectFromVpn` (NetdClient.h). A native core can `dlopen`/bind `protectFromVpn` and protect sockets entirely in native space without round-tripping to the Java `VpnService` object per socket.
- This is the established pattern for high-throughput native cores (avoids JNI upcalls on the hot connect path).
- Source: https://github.com/shadowsocks/shadowsocks-android/issues/2761 (Proposal: calling VpnService.protect directly via JNI; protectFromVpn in NetdClient)

Android memory posture (contrast with iOS)

- Android `VpnService` runs as a normal foreground service — **no hard 15/50 MiB jetsam cap** like iOS NE. Standard Android per-app memory / lowmemorykiller applies, far more generous. The severe memory constraint is an **iOS-only** design driver. (Engineering conclusion from the absence of any equivalent documented NE-style cap in the Android VpnService reference: https://developer.android.com/reference/android/net/VpnService)

---

4. Cross-platform design implications (synthesis)

1. iOS is the binding memory constraint. Design the shared native core to a hard steady-state budget of ~12–15 MB working set (NOT the 50 MiB headline) because (a) limits are undocumented and may be lower on older devices, (b) real 15 MB jetsam kills are reported on iOS 17, (c) spikes during high upload throughput are the dominant kill cause.
2. Prefer a WireGuard-class stateless/low-state datapath (no TCP connection tracking) to stay naturally small. Bound all buffer pools; size receive buffers to MTU+headroom.
3. Build the Rust core as a `staticlib` with `opt-level="z"/"s"`, `lto="fat"`, `codegen-units=1`, `panic="abort"`, `strip=true`; for the iOS target add nightly `build-std` + `panic_immediate_abort,optimize_for_size` for the largest

reduction. Smaller code = fewer mapped pages = lower NE footprint as well as smaller app.
4. Android core uses the same Rust staticlib over JNI: take the tun fd via `detachFd()`, read/write IP packets natively, and protect every outbound socket (ideally via native `protectFromVpn` to avoid per-socket JNI upcalls).
5. Always measure on a real device for both platforms; never trust the documented iOS number — instrument footprint and run upload-heavy stress (the speedtest scenario that triggers the sing-box/WireGuard kills).

NO single external doc covers the full Mullvad-parity mobile core; conclusions above are synthesized from the cited primary sources (Apple DTS threads, Rust official docs, Android reference, and maintained OSS cores). No fabricated figures — every number is cited or explicitly marked unofficial/UNCONFIRMED/ PENDING_FORENSICS.

Sources verified

All accessed 2026-06-25:

- https://developer.apple.com/forums/thread/73148 — Apple DevForums, Quinn DTS: packet tunnel = 50 MiB (iOS 16, reconfirmed 17/18); "do not hard code"; history.
- https://developer.apple.com/forums/thread/106377 — Apple DevForums, Quinn DTS: 15MB era; limit not officially documented; applies to whole process incl. libraries.
- https://developer.apple.com/forums/thread/747474 — Apple DevForums: observed 15 MB jetsam kill on iOS 17.3.1 despite 50 MiB docs (Feb 2024).
- https://developer.apple.com/forums/thread/680486 — Apple DevForums: max received datagram size in iOS packet tunnel (read-buffer sizing).
- https://github.com/SagerNet/sing-box/issues/3976 — iOS extension memory crash during SpeedTest on all WireGuard connections (spike-driven jetsam).
- https://groups.google.com/g/traffic-obf/c/PksmyfHMUb4 — Using Go in Apple network extensions (measuring memory via runtime.ReadMemStats).
- https://doc.rust-lang.org/rustc/codegen-options/index.html — rustc codegen options (opt-level, lto, panic, strip).
- https://nnethercote.github.io/perf-book/build-configuration.html — Rust Performance Book, build configuration (size profile, panic=abort, strip).
- https://markaicode.com/binary-size-optimization-techniques/ — Rust binary size: build-std, panic_immediate_abort, optimize_for_size (~43% reduction).

- https://blog.bitdrift.io/post/optimizing-rust-mobile-sdk-binary-size — bitdrift: optimizing Rust mobile SDK binary size (real-world).
- https://ospfranco.com/rust-reduce-binary-size/ — reducing Rust binary size (mobile/iOS).
- https://ospfranco.com/complete-guide-to-dylibs-in-ios-and-android/ — Rust dylib/staticlib for iOS & Android.
- https://developer.android.com/reference/android/net/VpnService — VpnService: protect(), establish(), packet I/O model.
- https://developer.android.com/reference/android/net/VpnService.Builder — Builder: addAddress/addRoute/setMtu/establish → ParcelFileDescriptor.
- https://github.com/xjasonlyu/tun2socks/issues/123 — passing the Android tun fd to native (detachFd).
- https://github.com/shadowsocks/shadowsocks-android/issues/2761 — calling VpnService.protect() directly via JNI (protectFromVpn / NetdClient).